

El lenguaje C y herramientas

El lenguaje C

C es un lenguaje de bajo nivel con abstracciones simple directas al hardware. Fue desarrollado en la década de 1970 por Dennis Ritchie. UNIX fue el primer SO escrito casi enteramente en C.

Tipos de datos

- Básicos
 - *Enteros* con diferente representación: `char` (un byte), `short int`, `int` y `long int`.

Pueden ser *signed* o *unsigned*.

Literales: `'A'` (*char*), `456` (*int*), `0367` (*int* en octal), `0x20AF` (*int* en hexadecimal).

- *Punto flotante*: `float`, `double` (simple y doble precisión).
- *Enumeraciones*: Dan identificadores a valores constantes.

c

```
enum day_of_week = {Sun, Mon, Tue, Wed, Thu, Fri, Sat};
```

Es equivalente a declarar `const int Sun = 0, Mon = 1, ...`

- Compuestos:
 - *Arreglos*: Elementos contiguos en memoria, indexados desde 0 a $N - 1$.
Ejemplo:

```
// ...
const int N = 100;
int a[N];
// access (indexing) operator
for (int i=0; i < N; i++) {
    a[i] = 0;
}
```

El tipo de `a` es `int *` y su valor es la dirección de su primer elemento.
El tipo de `a[0]` es `int` (su tipo base).

- *Strings*: Arreglos de caracteres (valores ASCII) terminados en 0 (`NULL`).
Ejemplos:

```
"It is a literal string" // anonymous (literal) string
```

```
char hello[6] = {'h', 'e', 'l', 'l', 'o', 0};
```

Ejercicio: Determinar el tipo de los símbolos `hello` y `hello[2]` .

- *Estructuras* o registros: Contiene *campos (fields)* de diferentes tipos.
Ejemplo:

```
/* type declaration */
struct person {
    int dni;
    char name[81];
    // ...
};

struct person p; // definición de la variable p
p.dni = 54786543;
strcpy(p.name, "Linus Torvalds");
// ...
```

- *Uniones disjuntas (union)*: Campos (*fields*) de diferentes tipos representados a partir de la misma dirección de memoria (*superpuestos*).
Permiten implementar valores *polimórficos*. Ejemplo:

c

```
// ip address type           in memory representation
union ip_address = {        // +-----+
    unsigned char v4[4];    // | v4[0], v6[0] |
    unsigned char v6[16];   // | v4[1], v6[1] |
};                          // |      ...      |
```

Las *constantes* se definen anteponiendo `const` al tipo.

Es posible definir un *alias* de un tipo usando `typedef`.

c

```
typedef byte unsigned char; // byte is alias of unsigned char
byte b1 = 0, b2 = 255;
```

Ejercicio: Definir un alias para declarar `person p;` (sin usar `struct`).

Punteros

Representan direcciones de memoria. Un *puntero a un valor de tipo T* se declaran de la forma `T *ptr;`. Tienen dos operadores asociados:

Referenciación y desreferenciación u obtener la dirección de.

c

```
int v = 10;
int *ptr = &v;           // ptr points to v = 10
printf("v = %d", *ptr); // output: v = 10
```

En una declaración de un *arreglo* de la forma `T v[N]`, `v` es un *puntero constante* a su primer elemento. O sea `*v == v[0]`.

Aritmética de punteros

Los punteros soportan los operadores aritméticos pero su semántica tiene en cuenta el *tamaño de la representación del tipo de dato*. Ejemplo:

```
int v[3] = {10, 20, 30};  
// ptr++ is actually the value of (unsigned int)(*ptr) += sizeof  
for (int *ptr = v; ptr < v + 3; ptr++) {  
    printf("%d ", *ptr);  
}
```

Punteros a funciones

El siguiente ejemplo muestra el uso de punteros y la posibilidad de implementar alto orden (pasar punteros a funciones como parámetros) en C.

```
int inc(int v, int step)  
{  
    return v + step;  
}  
  
// fptr_t is the type "pointer to a function of type int x int -  
typedef int(*fptr_t)(int, int);  
  
fptr_t ptr_inc = inc;  
printf("%d\n", ptr_inc(1, 2)); // output: 3
```

Funciones

Las funciones en C deben declarar el tipo del valor retornado. El tipo `void` representa *no valor* o *vacío*.

Los parámetros se pasan *por valor*. Se puede *simular* el pasaje por referencia usando punteros.

C soporta *funciones variádicas*, es decir con un número arbitrario de parámetros. Por ejemplo, la función de biblioteca tiene el perfil `int printf(const char *fmt, ...)`.

Para *invocar* a una función el compilador debe cononar el *perfil* para verificar los tipos de los argumentos y tipo de retorno.

Definición y declaración

Una *definición de una función* describe su comportamiento, es decir incluye su *cuerpo* (bloque de sentencias `{...}`).

Una *declaración* define sólo su perfil. Usadas generalmente cuando la función se define más abajo en el archivo fuente o en otro módulo.

Ejemplo:

```
#include <stdio.h> // incluye la cabecera con perfiles de funci

int d(int);        // declaración de g

void f(int x)
{
    return d(x) + 1;
}

int d(int v)
{
    return v * 2;
}
```

Para más detalles sobre sintaxis, sentencias y expresiones, ver [\[1\]](#)

Módulos

C no contiene sintaxis específica sobre módulos. Cada módulo de un programa se corresponde a un archivo fuente (`.c`). Cada módulo generalmente provee un archivo de cabecera (`.h`) conteniendo las *declaraciones* de sus funciones y tipos de datos provistos por el módulo para que sea incluido en otro módulo que lo use.

El módulo `main.c` *incluye* el módulo `hello.h` así puede usar las funciones y tipos definidos en `hello.c` .

```
// main.c                // hello.h                // hello.c
#include "hello.h"        char* hello();        char* hello(
```

```

int main()
{
    char *str = hello();
}
    {
        return "hi";
    }

```

El programa de arriba se compila con:

```
gcc -o myprog main.c hello.c
```

el cual genera el programa (*archivo ejecutable*) `myprog` y se puede ejecutar `myprog` mediante el comando `./myprog`.

Herramientas de desarrollo

Para el desarrollo de software en un lenguaje de programación se requiere de un conjunto de herramientas conocido como *toolchain* que contiene al menos:

- **Compilador:** Encargado de traducir *programas fuente* en *assembly*.
- **Ensamblador (assembler):** Genera *archivos objeto* (binarios) en un formato dado (ej: ELF).
- **Enlazador (linker):** Combina archivos objetos y *bibliotecas* para generar ejecutables o *bibliotecas*.

Nota: En el caso de los lenguajes C/C++ un programa fuente se *pre-procesa* para *expandir* las *macros* y *directivas* incluidas.

Dados los archivos fuente:

```

/* main.c */
#include <stdio.h> /* for printf() */

extern char* hello(void);

int main(void)
{
    printf("%s\n", hello());
}

```

```

    return 0;
}

```

y

c

```

/* hello.c */
#define hi "Hello world"

char *hello(void)
{
    return hi;
}

```

El siguiente diagrama muestra el proceso de compilación de la aplicación mediante el comando `gcc -o hello main.c hello.c`. Este comando genera el programa (archivo ejecutable) `hello` desde los programas fuente `main.c` y `hello.c`.

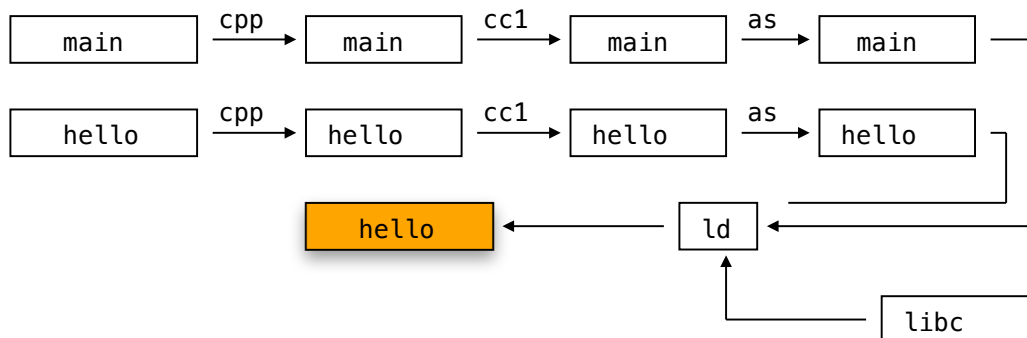


Figura 1: Pasos en la compilación de un programa C.

Como se puede apreciar en la figura 1, la compilación de un programa C consiste de varios pasos producidos por el comando `gcc`, el cual es un *driver* del proceso:

1. *Pre-procesamiento*: Se procesan las directivas del programa fuente y se *expanden* las macros encontradas. La salida es un archivo temporal (generalmente con extensión `.i`).
2. *Compilación*: El compilador (`cc1`) traduce de C a *assembly*.

3. *Assembler*: El *ensamblador* (`as`) produce los *archivos objeto* (binarios) correspondientes en el formato soportado por el SO como *ELF* en sistemas GNU-Linux).
4. *Enlazado (linking)*: El *linker* (`ld`) enlaza (combina y resuelve las referencias externas) de los archivos objeto y bibliotecas (en este ejemplo, la *biblioteca estándar* `libc.so` y genera el *ejecutable* final.

Estos pasos pueden ser controlados mediante el comando `gcc` . Los siguientes *flags* permiten detener en proceso en sus diferentes etapas:

- `-E` : Parar luego de la etapa de pre-procesamiento.
- `-S` : Parar luego de generar el *assembly*.
- `-c` : Parar luego de generar el archivo objeto (`.o`), es decir, no invocar al *linker*.

Archivos objeto

Un archivo objeto (`.o`) es un binario en el formato definido por la *ABI* del sistema. En sistemas GNU-Linux el formato es *ELF*, en Mac-OS es *Mach-O* y en MS-Windows son los formatos *Portable Executable (PE)* y *Common Object File Format (coff)* (ver [PE format](#)).

Si bien estos formatos difieren en detalles, un archivo objeto o ejecutable tienen las siguientes partes (denominados secciones y/o segmentos).

- Un *header* que describe el tipo de archivo, arquitectura, *entry point*, etc.
- Tipos de secciones:
 - *Data*: Valores de constantes y variables globales. Estas secciones pueden a su vez estar divididas en *datos inicializados* y *no inicializados*
 - *Text*: Instrucciones de programa
 - *Symbol table*: Tabla de símbolos que relacionan *identificadores* (funciones, variables, constantes, etc) con sus *direcciones de memoria*.
 - *Reallocation entries*: Direcciones de los operandos de las instrucciones en el programa que son *referencias externas*. Estas son generalmente operandos de las instrucciones de la forma (`call f` , `load/store d` , ...) donde `f` o `d` corresponden a direcciones no resueltas localmente en el módulo ya que refieren a símbolos externos.

Enlazado (linking)

El *linker* (o *link-editor*) es el encargado de producir los binarios (ejecutables y bibliotecas) en base a la *composición* de varios archivos objeto y bibliotecas.

Esta combinación de archivos binarios consiste de dos pasos:

1. Concatenar las secciones de código y datos de cada archivo objeto y *recalcular* las direcciones de las variables, constantes y funciones.
2. Resolver las direcciones de las instrucciones de llamado a funciones externas (del tipo `call <address>`) y lectura o escritura de variables globales externas.

Para resolver el paso 2, el linker usa las tablas de símbolos y de *reallocation entries* de cada archivo objeto.

La *symbols table* es un mapping de la forma

$\text{symtbl}(\text{symbol}) \rightarrow \text{address}$

que mapea cada símbolo a su dirección de memoria en el archivo objeto local.

La tabla de *relocation entries* es un mapping de la forma

$\text{relloc}(\text{instr_address}, \text{instr_type}) \rightarrow \text{external_symbol}$

que indica al linker qué operandos de la instrucción debe resolver ya que representa una *referencia externa*.

Dados los programas de ejemplo de arriba, el módulo `main.o` contiene dos referencias externas: `hello` y `printf`. Estos símbolos aparecen como *reallocation entries*. Es posible ver con `objdump -d main.o` el *disassembly* de las instrucciones en la sección de texto.

markup

```
.text section
_main:
00000000    ...
00000018    call 0x18          ; <----+
0000001c    call 0x1c          ; <-+  |
Reallocation Entries:      |  |
address      type      symbol ;  |  |
```

```

0x0000001c  call32  _printf ; --+ |
0x00000018  call32  _hello  ; -----+

```

Las instrucciones `call <address>` están sin resolver: `address` apunta a ellas mismas o 0, dependiendo del compilador. Estas instrucciones aparecen en la tabla de *reallocation entries* para que el linker las resuelva asociando con los símbolos a los que deberá resolver sus direcciones. En éste caso el símbolo `_hello` se resolverá durante el *linking estático* (ya que está definido en `hello.o`) y el `_printf` durante la ejecución (*linking dinámico*) ya que se encuentra definida en la biblioteca estándar que es una *shared library*.

Se puede ver que en los archivos objeto `main.o` y `hello.o`, las funciones `_main` y `_hello` tienen ambas dirección 0.

El comando `gcc -o myprog main.o hello.o` invoca al *linker* el cual enlaza los archivos objetos (y la biblioteca estándar) y genera el *ejecutable* `myprog`.

Al analizar la sección *text* de `myprog` (con `objdump -d myprog`) se puede observar que las direcciones de las instrucciones `call` se han resuelto.

markup

```

...
.interp section
"/lib/ld-linux.so"          ; dynamic linker (shared library)
.text section
main:
...
00003f50  call hello          ; call 0x3f68
00003f54  call 0x10003f74 ; printf() UNRESOLVED!
...
...          ret
hello:
00003f68  ...
...          ret

Symbol table:
...
hello 00003f68
...

Relocation entries:

```

```
.dynamic section
printf, call32, 0x00003f54 ; instruction to update dynamically
```

El *linker* combinó ambos módulos, recalculó direcciones (`_hello` tiene una dirección a continuación de la última instrucción de `_main`) y resolvió la dirección del `call` a la dirección de comienzo de `_hello` .

La invocación a `printf()` no se resuelve estáticamente, sino que se usará la información en la sección `dynamic` para resolverlos en tiempo de carga del programa usando el *dynamic linker* referenciado en la sección `interp` (*intérprete*). Se debe notar que el *dynamic linker* también es una *shared library* que el SO cargará la primera vez que se requiera.

Bibliotecas

Una biblioteca es una colección de módulos (código y datos) *reusables*.

Una *biblioteca* generalmente contiene un conjunto de archivos objeto (`.o`). Pueden ser de dos tipos:

1. Para *enlazado estático*
2. De *enlazado dinámico* (*shared*)

Una biblioteca de enlazado estático consiste en un *contenedor* (formato *archiver* en sistemas tipo UNIX) de archivos objetos. Un contenedor incluye un *índice* con los nombres de los archivos objeto que contiene.

En sistemas tipo UNIX se puede generar una biblioteca estática con el comando

```
markup
ar rcs libmylib.a file1.o file2.o ...
```

El comando `ar` es por *archiver*. Hacer `man ar` para más detalles.

Generalmente una biblioteca `B` se nombra de la forma `libB.a` .

Continuando con el ejemplo anterior, es posible crear `libhello.a` con

markup

```
ar rcs libhello.a hello.o
```

y crear el ejecutable `myprog` con

markup

```
gcc -o myprog -L $PWD -l hello
```

El flag `-L $PWD` adiciona el directorio corriente al *path* de búsqueda de `ld`.

Es posible generar la biblioteca `libhello.so` de enlazado dinámico haciendo

markup

```
gcc -c -fpic hello.c                # generate hello.o
gcc -shared -o libhello.so hello.o
```

La gran ventaja de las bibliotecas de enlazado dinámico (archivos con sufijo `.so` en GNU-Linux, `.DLL` en MS-Windows y `.dylib` en Mac-OS), es que los programas no incluyen todo el código que usarán sino que las bibliotecas se cargan y enlazan bajo demanda.

La desventaja es que se produce una pequeña sobrecarga en ejecución ya que se debe realizar el enlazado antes de comenzar la ejecución de cada proceso.

Veremos que esta *demora* se puede reducir significativamente con *lazy linking*.

Es posible que una biblioteca tenga sus dos versiones: estática y dinámica. Generalmente el linker prefiere la versión dinámica a menos que se especifique el nombre específico (ej: `hello.a`).

Lazy linking

Muchos sistemas operativos modernos realizan *lazy linking*, es decir en lugar de resolver *todas* las referencias externas, se enlaza cada función en su primera invocación. Esto reduce la *latencia* inicial en la ejecución del proceso.

Con *lazy linking* el linker (estático) genera por cada *función externa* una pequeña rutina (*stub*) en el ejecutable la cual es responsable de resolver ese símbolo de función en su primera invocación usando el linker dinámico.

Por ejemplo, en una llamada de la forma `call printf` se invoca a `printf_stub`. Esta rutina iniciará la resolución de la dirección de la primera instrucción de `printf` que está en la biblioteca estándar (compartida por todos los procesos, cargada una vez en la memoria).

Comúnmente esta maquinaria se implementa con una *tabla indirecta* llamada *Global Offset Table (GOT)* como se muestra a continuación.

Inicialmente, la `GOT[printf]` apunta a una instrucción de `printf_stub` que invocará al *linker*.

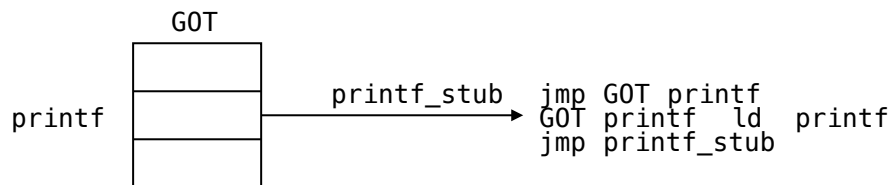


Figura 2: GOT antes de invocar a `printf_stub`.

Cuando el programa invoca a `printf`, saltará a `printf_stub`, la cual invocará al linker dinámico para que resuelva el símbolo. El valor retornado (dirección de `printf`) se almacena en `GOT[printf]`, como se muestra a continuación.

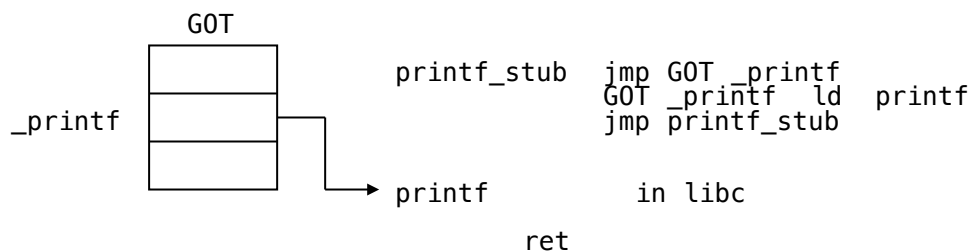


Figura 3: GOT luego de la primera invocación a `printf_stub`.

En las siguientes invocaciones del programa a `printf_stub` ésta salta a `printf` (mediante su primera instrucción). La sobrecarga es que en cada invocación se requiere un salto adicional.

La función `printf` retorna al punto de continuación del programa invocante.

Importante

Notar que los *stubs* usan instrucciones de salto incondicionales (`jmp`) y no usan `call` ya que se debe retornar al programa invocante (no al *stub*).

Build systems

Un sistema de desarrollo incluye herramientas de automatización de los pasos de producción del software. Para la construcción de un componente de software se debe tener en cuenta sus dependencias y los comandos a aplicar en cada paso del proceso de compilación y enlazado hasta obtener los programas y/o bibliotecas finales.

GNU build system

Este sistema^[1] se basa en la herramienta `make`, la cual se basa en una especificación de los objetos o *targets* a construir y sus dependencias. La especificación se escribe en un archivo `Makefile` y consiste en una secuencia en reglas de la forma:

```
target: dependencies
    <tab> build command
```

markup

A modo de ejemplo, el programa `myprog` basado en los archivos fuente del capítulo anterior, puede especificarse en la siguiente regla:

```
myprog: main.o hello.o
    gcc -o myprog main.o hello.o

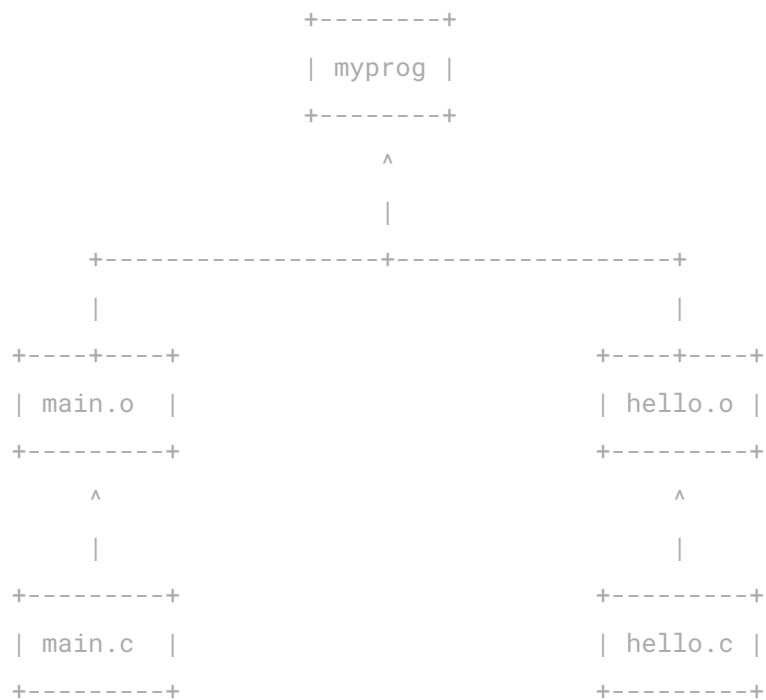
main.o: main.c
    gcc -c main.c

hello.o: hello.c
    gcc -c hello.c
```

Makefile

La primera regla determina que para generar el ejecutable `myprog`, el cual depende de los archivos objeto `main.o` y `hello.o`, se construye con el comando `gcc -o $@ $? .`.

El programa `make` construye un grafo de dependencias para determinar qué debe *reconstruirse* dependiendo del *timestamp* (fecha+hora) de los archivos.



Por ejemplo, si el archivo `main.o` no existe o *es mas viejo* que `main.c` , deberá dispararse la segunda regla para reconstruir `main.o` . Luego, se deberá ejecutar nuevamente la primera regla en la cual se ejecuta el linker para generar el ejecutable final.

Estas reglas pueden escribirse de manera mas compacta usando *variables automáticas*.

Makefile

```

myprog: main.o hello.o
    gcc -o $@ $^

%.o: %.c
    gcc -c $<
  
```

Para más detalles sobre las *variables* usadas ver [automatic variables](#).

El *GNU build system* incluye otras herramientas que permiten automatizar el proceso de compilación y enlazado de paquetes de software en forma independiente de la plataforma. Estas herramientas se conocen como las *autotools* que contiene los siguientes componentes:

- *Autoconf*: Crea archivos de configuración y el script `configure` desde la especificación `autoconf.ac`.
- *Automake*: Se basa en las directivas en `Makefile.am` y `configure.ac` y genera el `Makefile.in` que será usado por `make`.

Con estas herramientas un desarrollador genera un *paquete* `my-package.tgz` (contenedor *tar* comprimido). [Aquí](#) se muestra un ejemplo de cómo crear un paquete simple.

Este paquete se distribuye y un usuario lo puede instalar siguiendo los siguientes pasos:

```
# tar xzvf my-package.tgz
# ./configure && make && sudo make install
```

sh

Referencias

[1]



The C programming Language. Brian Kernighan and Dennis Ritchie. 2nd Ed. Prentice Hall.

[2]



GNU build system:

https://www.gnu.org/software/automake/manual/html_node/GNU-Build-System.html

Anterior

Interfaces

Próximo

Procesos y threads